IoT Environmental Monitoring System

Final Project Report

1. Introduction

This project implements an IoT environmental monitoring system using MQTT, Python, MongoDB, MySQL, and Neo4j. The system simulates IoT devices that publish environmental and network data through MQTT topics. A Python subscriber receives the MQTT messages, processes the incoming JSON data, generates alerts, validates measurements, and routes information to different database systems based on the topic and type of data.

2. Project Objectives

• Implement MQTT-based communication.

• Process environmental sensor data in real time.

• Use topic-based routing for data processing.

• Integrate MongoDB, MySQL, and Neo4j in one architecture.

• Generate alerts for abnormal sensor readings.

• Model network topology using graph relationships.

3. Technologies Used

Technology	Purpose
Python	Main programming language
MQTT	Communication protocol
Mosquitto	MQTT broker
MongoDB	Flexible document storage
MySQL	Structured relational storage
Neo4j	Graph relationship modeling
Docker	Containerized infrastructure

4. System Architecture

The architecture uses MQTT publishers, a Mosquitto broker, and a Python subscriber processing engine. Data is routed based on MQTT topics. Routing Logic: - sensors/environment → MongoDB + MySQL - sensors/network → MongoDB + Neo4j

```
Environment Publisher
        ↓
MQTT Broker (Mosquitto)
        ↓
Python Subscriber
        ↓
MongoDB + MySQL

Network Publisher
```

```
        ↓
MQTT Broker (Mosquitto)
        ↓
Python Subscriber
        ↓
MongoDB + Neo4j
```

5. MongoDB Enriched Event Storage

MongoDB stores enriched environmental event documents. The documents contain nested sensor readings, alert arrays, threshold values, timestamps, and metadata. The alert system generates warnings for: - high temperature - high humidity - poor air quality

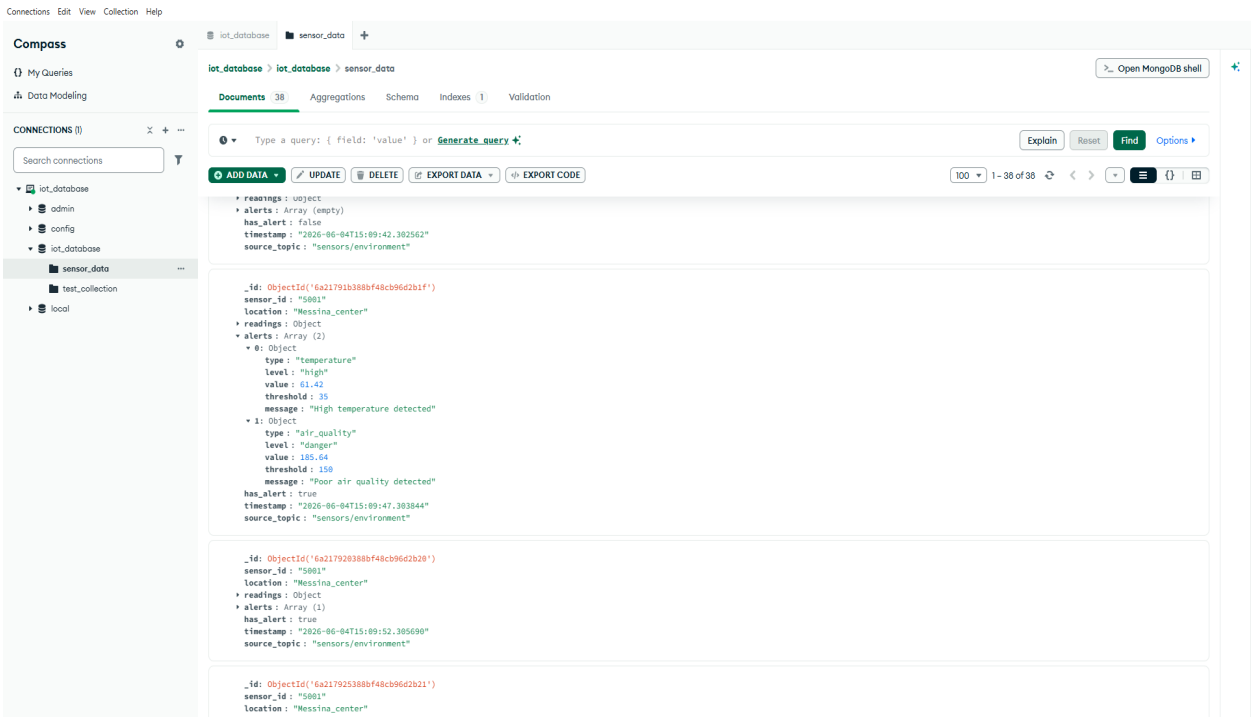

Figure 1: MongoDB enriched alert document.

6. MySQL Structured Storage

MySQL stores validated environmental measurements in a structured relational table. Only valid data is inserted into MySQL after required-field and range validation. The table stores: - sensor ID - location - temperature - humidity - air quality - timestamp

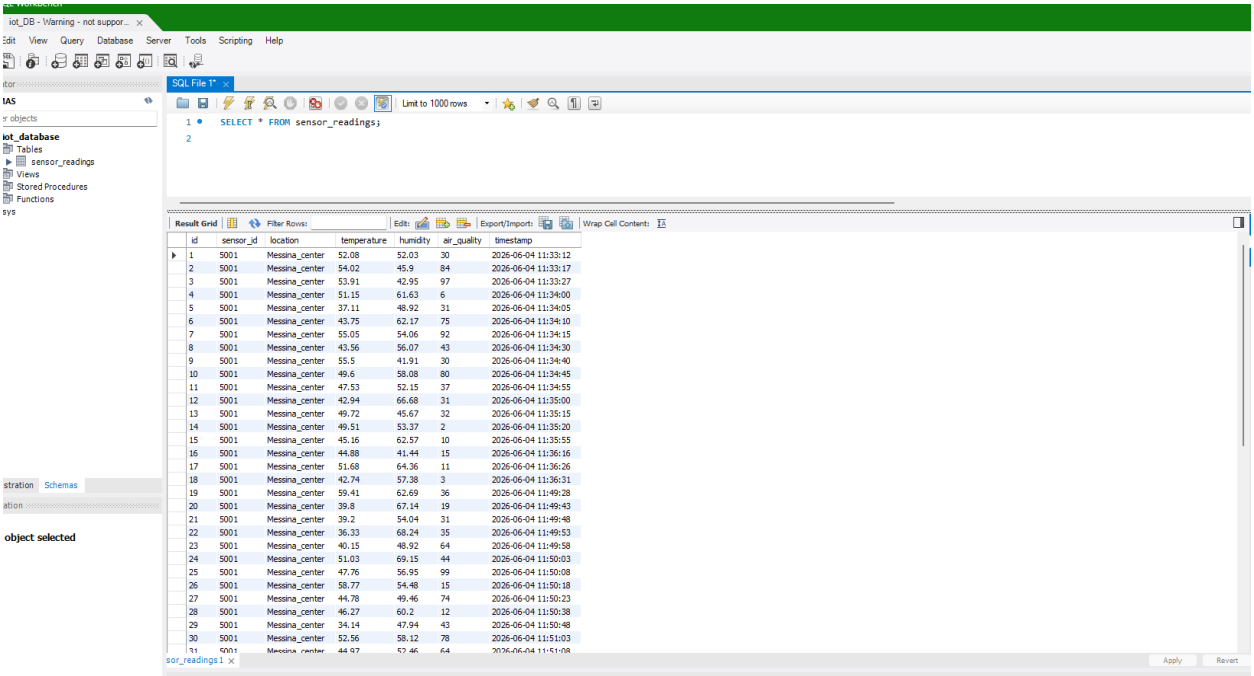

Figure 2: MySQL sensor_readings table.

7. Neo4j Graph Topology

Neo4j stores relationship-based network topology data. The graph models: - Sensor nodes - Gateway nodes - Location nodes - CONNECTED_TO relationships - LOCATED_IN relationships This allows the system to represent network connections visually.

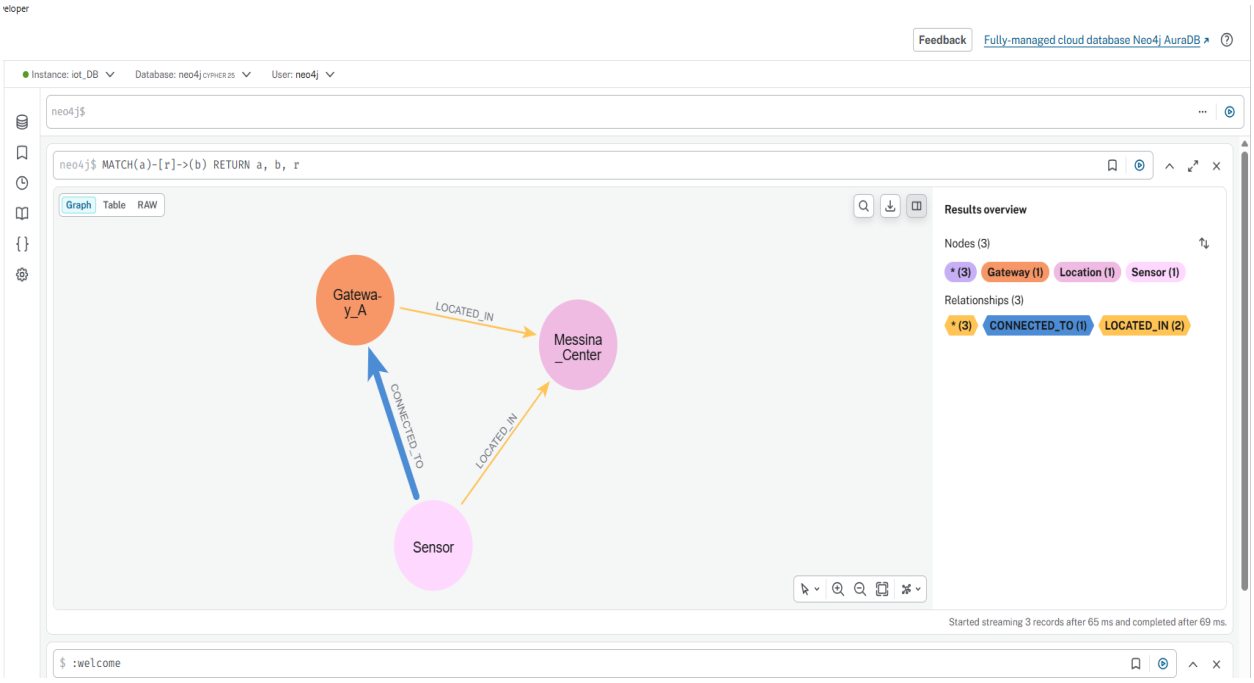

Figure 3: Neo4j graph relationships.

8. Subscriber Processing Engine

The Python subscriber acts as the central processing engine. Responsibilities: - receive MQTT messages - process JSON data - generate alerts - validate measurements - route data to appropriate databases

Figure 4: Subscriber processing output.

9. Conclusion

The project successfully demonstrated MQTT communication, topic-based routing, multi-database integration, alert generation, graph relationship modeling, and event-driven processing. The architecture uses polyglot persistence by combining MongoDB, MySQL, and Neo4j, each optimized for different types of data.

Author: Alex Demse

University of Messina - Data Analysis Program